

DFS, BFS & Recursion — Complete Study Guide

Transcribed from handwritten/annotated notes

DFS & BFS — Complete Guide

1. What is it?

DFS (Depth-First Search) and BFS (Breadth-First Search) are the two fundamental traversal techniques for graphs and trees.

- **DFS** goes as deep as possible down one path before backtracking — implemented via recursion or an explicit stack.
- **BFS** explores level by level, visiting all neighbors at the current distance before going further — implemented via a queue.

Both visit every reachable node exactly once (with a visited set), but the **order** and the **guarantees** differ, which is what determines which one you need.

2. Why is it used?

- Systematic way to explore all nodes/paths in a graph or tree without missing or repeating any
- BFS gives the **shortest path in an unweighted graph** for free, because it explores in increasing distance order
- DFS naturally expresses **exhaustive exploration** (all paths, all combinations, backtracking) and **structural properties** (cycles, connectivity, topological order)
- Both run in $O(V + E)$ — visiting every vertex and edge once — which is optimal for graph traversal

3. Where is it used?

DFS: Path existence / all paths between two nodes • Cycle detection (directed and undirected graphs) • Topological sort • Connected components / islands (grid flood fill) • Backtracking: permutations, combinations, subsets, N-Queens, Sudoku • Tree traversal (preorder, inorder, postorder)

BFS: Shortest path in unweighted graph / grid • Level-order tree traversal • Minimum number of steps/moves (word ladder, sliding puzzle) • Multi-source spreading (rotting oranges, walls and gates) • Bipartite graph checking (via 2-coloring while traversing levels)

4. How to identify a DFS vs BFS problem (pattern triggers)

Signal in the problem statement	What it suggests
"shortest path", "minimum steps/moves", "fewest number of..."	BFS — guarantees shortest path in unweighted graphs
"all paths", "does a path exist", "explore every possibility"	DFS — exhaustive exploration
"level order", "print level by level", "distance from source"	BFS — processes nodes in distance order
"connected components", "number of islands", "flood fill"	Either works — DFS is usually simpler to write (recursion)
"cycle detection"	DFS — track recursion stack (directed) or parent (undirected)
"topological order", "course schedule", "build order"	DFS (postorder + reverse) or BFS (Kahn's algorithm)
"generate all subsets/permutations/combinations"	DFS (backtracking) — BFS doesn't naturally express choice-branching with undo
"nearest", "closest", "minimum time to reach all nodes"	BFS, often multi-source
Weighted graph + "shortest path"	Neither plain DFS nor BFS — needs Dijkstra/Bellman-Ford instead

Paraphrase test: If the problem asks “what is the minimum distance/steps” in an unweighted setting — BFS. If it asks “does X exist”, “find all X”, or “can you reach” without needing the shortest route — DFS.

5. Can you tell from constraints alone?

Yes, partially:

- Grid/graph size up to 10^4 - 10^6 nodes/cells with expected $O(V+E)$ → either DFS or BFS traversal is intended; the choice between them comes from what's being asked (shortest vs exhaustive), not from constraints.
- If recursion depth could exceed $\sim 10^4$ (deep/skewed trees, long chains), a recursive DFS risks stack overflow in Java — prefer iterative DFS with an explicit stack, or use BFS instead.
- If the problem says “minimum” or “shortest” AND edges are unweighted (all cost 1) → BFS, not DFS, even though DFS could technically find a path — it won't guarantee the shortest one without extra bookkeeping.
- If weights are involved despite looking like a shortest-path problem → neither DFS nor BFS is sufficient; that's a Dijkstra/Bellman-Ford signal.

6. Intuition / Rationale

Why BFS finds the shortest path: BFS processes nodes in strict order of distance from the source — it fully exhausts all nodes at distance d before touching any node at distance $d+1$. The first time you reach the target, it's guaranteed to be via the shortest possible number of edges, because no longer path could have reached it first.

Why DFS is natural for exhaustive search: DFS's call stack IS the current path being explored. This makes it trivial to backtrack (undo the last choice and try another) — which is exactly what problems like permutations, N-Queens, and path-finding-with-constraints need. BFS's queue doesn't preserve “the current path” in the same accessible way, making backtracking awkward.

Why both need a visited set: without it, cyclic graphs cause infinite loops — you'd keep re-visiting the same nodes forever. Trees don't strictly need one (no cycles), but graphs always do.

7. Time & Space Complexity (generic)

- **Time:** $O(V + E)$ for both — every vertex visited once, every edge examined once
- **Space:**
 - DFS: $O(V)$ for the visited set + $O(H)$ for the recursion/explicit stack, where H is the max depth (worst case $O(V)$ for a skewed graph)
 - BFS: $O(V)$ for the visited set + $O(W)$ for the queue, where W is the max width of a level (worst case $O(V)$)

8. Types of DFS & BFS (with Java templates)

Type A — Recursive DFS (graph/tree traversal)

Used for: path existence, connected components, general traversal.

```
public void dfs(int node, Map<Integer, List<Integer>> graph, Set<Integer> visited) {
    if (visited.contains(node)) return;
    visited.add(node);
    // process(node) here

    for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
        dfs(neighbor, graph, visited);
    }
}
```

Type B — Iterative DFS (explicit stack, avoids recursion depth limits)

Used for: same as Type A, but safe for very deep/large graphs.

```

public void dfsIterative(int start, Map<Integer, List<Integer>> graph) {
    Set<Integer> visited = new HashSet<>();
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);

    while (!stack.isEmpty()) {
        int node = stack.pop();
        if (visited.contains(node)) continue;
        visited.add(node);
        // process(node) here

        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited.contains(neighbor)) stack.push(neighbor);
        }
    }
}

```

Type C — BFS (shortest path / level order)

Used for: shortest path in unweighted graph, level-order traversal, min steps.

```

public int bfsShortestPath(int start, int target, Map<Integer, List<Integer>> graph) {
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(start);
    visited.add(start);
    int steps = 0;

    while (!queue.isEmpty()) {
        int size = queue.size(); // process one full level at a time
        for (int i = 0; i < size; i++) {
            int node = queue.poll();
            if (node == target) return steps;

            for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    queue.offer(neighbor);
                }
            }
        }
        steps++; // moved to the next level/distance
    }
    return -1; // target not reachable
}

```

Type D — Grid DFS (flood fill / connected regions)

Used for: number of islands, flood fill, max area of island.

```

public int numIslands(char[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    boolean[][] visited = new boolean[rows][cols];
    int count = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1' && !visited[r][c]) {
                dfsGrid(grid, visited, r, c);
                count++; // found a new island, explore all of it
            }
        }
    }
    return count;
}

private void dfsGrid(char[][] grid, boolean[][] visited, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (visited[r][c] || grid[r][c] == '0') return;
    visited[r][c] = true;

    dfsGrid(grid, visited, r + 1, c);
    dfsGrid(grid, visited, r - 1, c);
    dfsGrid(grid, visited, r, c + 1);
    dfsGrid(grid, visited, r, c - 1);
}

```

Type E — Multi-Source BFS

Used for: rotting oranges, walls and gates, spreading from multiple starting points simultaneously.

```

public int multiSourceBFS(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> queue = new LinkedList<>();
    int freshCount = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) queue.offer(new int[]{r, c}); // seed ALL sources at once
            else if (grid[r][c] == 1) freshCount++;
        }
    }

    int minutes = 0;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!queue.isEmpty() && freshCount > 0) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int[] cell = queue.poll();
            for (int[] d : dirs) {
                int nr = cell[0] + d[0], nc = cell[1] + d[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] == 1) {
                    grid[nr][nc] = 2;
                    freshCount--;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        minutes++;
    }
    return freshCount == 0 ? minutes : -1; // -1 if some fresh cells unreachable
}

```

Type F — Backtracking DFS (explicit undo)

Used for: permutations, combinations, subsets, N-Queens.

```

public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, new ArrayList<>(), new boolean[nums.length], result);
    return result;
}

private void backtrack(int[] nums, List<Integer> current, boolean[] used, List<List<Integer>> result) {
    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current)); // copy - current will keep mutating
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        current.add(nums[i]);

        backtrack(nums, current, used, result); // explore

        current.remove(current.size() - 1); // undo (the "back" in backtracking)
        used[i] = false;
    }
}

```

9. Comparison of the patterns

Type	Structure	Typical trigger words	Complexity
A. Recursive DFS	call stack	"traverse", "path exists", "connected"	$O(V+E)$
B. Iterative DFS	explicit Deque as stack	same as A, but very deep/large graphs	$O(V+E)$
C. BFS	Queue, level-by-level	"shortest path", "min steps", "level order"	$O(V+E)$
D. Grid DFS	recursion over 2D array	"islands", "flood fill", "connected region"	$O(\text{rows} \times \text{cols})$
E. Multi-source BFS	queue seeded with multiple starts	"simultaneously spread", "min time to reach all"	$O(\text{rows} \times \text{cols})$
F. Backtracking DFS	recursion + explicit undo step	"all permutations/combinations/subsets"	$O(\text{branching}^{\text{depth}})$

Key distinguishing question: Do I need the shortest/minimum distance ($\rightarrow$ C or E, BFS), am I exploring a grid region ($\rightarrow$ D), am I generating all valid arrangements with undo ($\rightarrow$ F), or is it general reachability/structure ($\rightarrow$ A/B, DFS)?

10. Quick decision checklist (use this in an interview)

1. Does the problem need shortest path / minimum steps in an unweighted graph? $\rightarrow$ BFS (Type C)

2. Are there multiple simultaneous starting points spreading outward? → Multi-source BFS (Type E)
3. Do I need to generate all valid combinations/arrangements, with the ability to undo a choice? → Backtracking DFS (Type F)
4. Is it a 2D grid asking about connected regions/islands? → Grid DFS (Type D) (BFS works too, DFS is usually simpler to write)
5. Is the graph very deep or could cause a stack overflow with recursion? → Iterative DFS (Type B)
6. Otherwise — general traversal, cycle detection, topological sort → Recursive DFS (Type A)
7. Are edges weighted and you still need shortest path? → Neither — use Dijkstra (non-negative weights) or Bellman-Ford (negative weights allowed)

11. Comparison with Two Pointers, Sliding Window, Sweep Line, and Greedy

	Two Pointers / Sliding Window / Sweep Line / Greedy	DFS / BFS
Data shape	Linear (array, sorted events, intervals)	Graph/tree structure (nodes + edges, possibly a grid)
Core mechanism	Pointer(s) advancing through a sequence with a monotonic invariant	Traversal following edges/adjacency, with branching
Backtracking	None — a discarded position is never revisited	DFS backtracking explicitly re-visits/undoes choices
When they meet	Grid BFS/DFS problems sometimes combine with a sweep-like "process by distance/level" idea (Type E is essentially a sweep line over graph distance)	—

The core difference: earlier patterns (two pointers, sliding window, sweep line, greedy) all operate on **flat, ordered sequences** where you never branch. DFS/BFS operate on **branching structures** — there isn't one path forward, there are potentially many, and the choice of DFS vs BFS is about which order you explore those branches in.

Recursion — Complete Guide

1. What is it?

Recursion is when a function **calls itself** to solve smaller instances of the same problem, until it reaches a **base case** simple enough to answer directly. The results of the smaller calls are then combined to build the answer to the original problem.

Every recursive function needs exactly two parts: 1. **Base case(s)** — the condition(s) where the function stops calling itself and returns directly. 2. **Recursive case** — where the function calls itself on a smaller/simpler version of the input, and combines that result into the current answer.

2. Why is it used?

- Naturally expresses problems that are **self-similar** — the solution to size n is built from the solution to a smaller size
- Often dramatically simpler to read/write than the equivalent iterative version (tree traversals, backtracking, divide and conquer)
- The call stack gives you “free” backtracking — undoing a choice is often just returning from the call
- It's the natural language for problems defined recursively in the first place (trees, nested structures, mathematical recurrences)

3. Where is it used?

- Tree and graph traversal (DFS is inherently recursive)
- Divide and conquer (merge sort, quick sort, binary search)
- Backtracking (permutations, combinations, subsets, N-Queens, Sudoku)
- Dynamic programming (top-down / memoized recursion)
- Mathematical recurrences (factorial, Fibonacci, power, GCD)
- Nested/recursive data structures (nested lists, JSON parsing, file system traversal)
- String/array problems expressible as “solve for the rest after handling the first/last element”

4. How to identify a Recursion problem (pattern triggers)

Signal in the problem statement	What it suggests
"tree", "nested", data structure defined in terms of itself	Recursion mirrors the data's own definition
"all possible ways to...", "generate all..."	Recursion + backtracking
Problem naturally splits into "solve for smaller input, then combine"	Divide and conquer recursion
"the answer for n depends on the answer for n-1 (or smaller)"	Direct recursive recurrence
You can describe the solution as "do X, then recursively solve the rest"	Strong recursion signal
Problem involves undo/backtrack after trying a choice	Recursion (backtracking specifically)

5. Can you tell from constraints alone?

Yes, partially:

- Small n ($\leq \sim 20-25$) with an expected exponential-looking solution ("all subsets", "all permutations") → recursion/backtracking is expected; brute-force enumeration is intentional at that size.
- Large n (10^4+) with a naive recursive recurrence and overlapping subproblems (e.g. Fibonacci) → plain recursion will time out; you need memoization (top-down DP) or an iterative bottom-up version.
- If recursion (chain-like structures, n up to 10^4-10^5) → risk of `StackOverflowError` in Java; prefer converting to iteration or increasing stack size deliberately, don't assume recursion is safe just because it's simple.
- If overlapping subproblems exist (the same sub-call happens many times with identical inputs) → recursion alone is exponential; add memoization.

Paraphrase test: If you can finish the sentence "the answer to a problem of size n is [some combination of] the answer(s) to problem(s) of smaller size" — that's a recursive definition, and the code should mirror it directly.

6. Intuition / Rationale

Recursion works because of **mathematical induction**: if you can prove the base case is correct, and prove the recursive case is correct *assuming* smaller sub-calls are correct, then by induction the function is correct for all valid inputs. You don't need to trace every recursive call by hand — you only need to trust the recursive leap of faith once the base case and the inductive step are both verified.

The "leap of faith": when writing `return solve(n-1) + something;`, you should not mentally unroll what `solve(n-1)` does internally — you should *assume it's already correct* (by the inductive hypothesis) and focus only on how to correctly build the correct result for n from that assumed-correct result. This is the single biggest mental shift that makes recursion easy to write instead of confusing.

7. Time & Space Complexity (generic)

Why the call stack matters: each recursive call gets its own stack frame with its own local variables. This is what makes backtracking trivial — when a call returns, its local state (e.g. `current.add(x)` before the call, `current.remove(x)` after) naturally “undoes” as control returns to the caller, without needing to manually manage a global undo stack.

- **Time:** depends entirely on the recurrence relation.
 - One recursive call per level, shrinking input by a constant (e.g. $n-1 \rightarrow O(n)$)
 - Two recursive calls per level, shrinking input by half (e.g. binary search) $\rightarrow O(\log n)$
 - One call, shrinking input by half (e.g. merge sort) $\rightarrow O(n \log n)$
 - Two calls, each on half the input (e.g. naive Fibonacci) $\rightarrow O(2^n)$
- **Space:** $O(\text{depth of recursion})$ for the call stack, **regardless of how many total calls are made** — this is the most commonly mis-estimated part. Naive Fibonacci makes $O(2^n)$ calls but only ever has $O(n)$ stack frames alive at once, so its space complexity is $O(n)$, not $O(2^n)$.

8. Types of Recursion (with Java templates)

Type A — Linear Recursion (single recursive call)

Used for: factorial, sum of array, simple traversal of a linked structure.

```
public int factorial(int n) {
    if (n <= 1) return 1; // base case
    return n * factorial(n - 1); // recursive case: trust factorial(n-1) is correct
}
```

Type B — Divide and Conquer (multiple calls on disjoint sub-inputs)

Used for: merge sort, quick sort, binary search, tree height/depth.

```
public void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return; // base case: 0 or 1 elements, already sorted
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid); // solve left half
    mergeSort(arr, mid + 1, right); // solve right half
    merge(arr, left, mid, right); // combine the two sorted halves
}
```

Type C — Tree Recursion (mirrors the tree's own structure)

Used for: tree traversals, tree depth, validate BST, tree sum.

```

public int maxDepth(TreeNode root) {
    if (root == null) return 0; // base case: empty tree has depth 0
    int leftDepth = maxDepth(root.left); // trust this is correct (leap of faith)
    int rightDepth = maxDepth(root.right); // trust this is correct too
    return 1 + Math.max(leftDepth, rightDepth); // combine: this node + deeper subtree
}

```

Type D — Backtracking (explore, then undo)

Used for: permutations, combinations, subsets, N-Queens, word search.

```

public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current)); // every state along the way is a valid subset
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]); // choose
        backtrack(nums, i + 1, current, result); // explore
        current.remove(current.size() - 1); // un-choose (backtrack)
    }
}

```

Type E — Memoized Recursion (top-down DP)

Used for: Fibonacci, climbing stairs, any recursion with overlapping subproblems.

```

public long fib(int n, Map<Integer, Long> memo) {
    if (n <= 1) return n; // base case
    if (memo.containsKey(n)) return memo.get(n); // avoid recomputing a solved subproblem

    long result = fib(n - 1, memo) + fib(n - 2, memo);
    memo.put(n, result); // cache before returning
    return result;
}

```

Type F — Mutual/Indirect Recursion (two functions call each other)

Used for: parsing recursive grammars, alternating state machines (less common in interviews, but appears in parsers).

9. Comparison of the patterns

Type	Number of recursive calls	Typical trigger words	Complexity driver
A. Linear	1 call, shrinks by constant	"compute for n based on n-1"	O(n) time, O(n) space
B. Divide and conquer	2+ calls on disjoint sub-ranges	"sort", "search", "split problem in half"	O(n log n) typical
C. Tree recursion	1 call per child (2 for binary tree)	"tree", "depth", "traverse"	O(n) time (n = node count), O(h) space
D. Backtracking	Variable — one call per choice, with undo	"all permutations/combinations/subsets"	O(branching^depth), often exponential
E. Memoized	Same shape as A/C but cached	overlapping subproblems, "number of ways"	O(number of unique subproblems)
F. Mutual	Alternates between two+ functions	grammar/parsing problems	depends on recurrence

10. Quick decision checklist (use this in an interview)

1. Can I state the base case(s) first — the smallest input(s) I can answer without recursing? Always start here.
2. Does my recursive case call the function on a strictly smaller input, guaranteeing eventual termination? If not, infinite recursion.
3. Am I trusting the recursive call's result (leap of faith) instead of trying to mentally trace every nested call?
4. Do I need to **undo** a choice after exploring it (add to a list, then remove)? → **Backtracking (Type D)**
5. Will the same sub-call happen with identical arguments more than once? → add a **memo map (Type E)** before it becomes exponential
6. Is the recursion depth potentially very large (deep chains, large n)? → consider **converting to iteration** to avoid `StackOverflowError`

7. Does the problem split cleanly into independent halves that don't need to “know” about each other until combined? → **Divide and conquer (Type B)**

11. Comparison with DFS/BFS and the other patterns

	Recursion (general)	DFS	Backtracking
Relationship	The general mechanism	A specific application of recursion to graph/tree traversal	A specific application of recursion with explicit choose/explore/un-choose
What's being explored	Any self-similar sub-problem	Nodes/edges of a graph or tree	A decision tree of choices (not necessarily a “real” graph)
Undo needed?	Only if backtracking	Rarely (just mark visited)	Always — that's the defining feature

The cleanest mental model: recursion is the umbrella technique. DFS is recursion specifically applied to graph/tree adjacency. Backtracking is recursion specifically applied to a decision tree where you must undo a choice to try the next one. Divide and conquer is recursion where sub-calls are on independent, non-overlapping pieces of input that get merged afterward. Memoized recursion (top-down DP) is recursion plus a cache, used when sub-calls overlap. Once you see recursion as the shared foundation, DFS/backtracking/divide-and-conquer/DP all become “recursion + one extra rule,” not separate things to memorize from scratch.